ОТЧЕТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

по теме: “Разработка интеллектуального персонального ассистента Delëz для самоанализа и визуализации эмоциональных состояний”

Шифр темы: ?

Номер государственной регистрации: ?

Инвентарный номер: ?

Руководитель НИР
Менеджер проекта
_____ Е.А. Котельникова

(подпись, дата)

Исполнители:

Фамилия И.О.	**Роль**	**Разделы (задачи)**
Селиванов Матвей Сергеевич	Пентестер, DevOps	Настройка SonarQube, CI/CD, тестирование на проникновение
Васильев Данил Вячеславович	Фронтент-разработчик	Проектирование БД, API, голосовой ввод (Whisper), рефакторинг, тесты
Титов Даниил Романович	Архитектор БД	Настройка PostgreSQL, RLS, управление доступом
Еремеев Иван Иванович	Бэкенд-разработчик	RAG, LangGraph, векторные БД, выбор модели LLM
Гущин Даниил Артёмович	DevOps	Настройка серверов, GitLab, delez.tech, Nginx, Docker

РЕФЕРАТ

Объект разработки – интеллектуальный персональный ассистент «Delëz», предназначенный для ведения дневника самонаблюдения и анализа эмоциональных состояний на основе философской концепции ризомы Жиля Делёза.

Цель работы – создание программного продукта, позволяющего пользователю фиксировать события, эмоции и выводы, автоматически выявлять связи между ними и визуализировать их в виде графа, а также отслеживать изменения во времени.

Методы и инструменты – при разработке применялись методы машинного обучения (GigaChat, RAG, LangGraph), язык Python (FastAPI) для бэкенда, TypeScript (React) для фронтенда, системы управления базами данных PostgreSQL, Neo4j, ChromaDB, контейнеризация Docker, системы непрерывной интеграции GitLab CI и статического анализа SonarQube.

Результаты – разработано веб-приложение с возможностью голосового ввода (Whisper), построения интерактивного графа «ризомы», проведения экспериментов над гипотезами о себе. Реализована Zero-Knowledge Architecture для обеспечения приватности данных. Проведено модульное, интеграционное и стресс-тестирование, достигнуто покрытие кода более 80%. Обеспечена безопасность на уровне Docker и приложения.

Новизна – подход заключается в использовании философской модели ризомы для организации персональных данных и адаптивных ответов LLM с учётом истории пользователя, а также в гибридном поиске (граф + векторные эмбеддинги) для выявления скрытых паттернов.

Область применения – психологическая самопомощь, коучинг, ведение личных дневников, исследования в области эмоционального интеллекта.

Степень внедрения – продукт готов к опытной эксплуатации, развёрнут на домене delez.tech.

Эффективность – социальная значимость заключается в предоставлении пользователям инструмента для глубинного самоанализа, снижения тревожности и повышения осознанности.

Прогноз развития – планируется интеграция с мобильными приложениями, расширение пула LLM, внедрение рекомендательной системы на основе анализа графа.

Ключевые слова: персональный ассистент, ИИ, ризома, анализ эмоций, граф событий, GigaChat, RAG, LangGraph, векторная БД, дневник самонаблюдения, Python, Docker, CI/CD, тестирование.

1 Аналитическая часть 5
1.1 Функциональные требования 5
1.2 Нефункциональные требования 6
1.3 Сценарии использования 6
2 Проектирование архитектуры 8
2.1 Общая архитектура системы 8
2.2 Компоненты и их взаимодействие 9
2.3 Схема базы данных 9
2.4 Проектирование интерфейса 10
3 Реализация 11
3.1 Используемые технологии и инструменты 11
3.2 Описание модулей 11
3.3 Особенности реализации ключевых функций 12
4 Тестирование 14
4.1 Модульное тестирование 14
4.2 Интеграционное тестирование 14
4.3 Нагрузочное тестирование 14
4.4 Безопасность и статический анализ 15
5 Интеграция и сопровождение 16
5.1 Контейнеризация и оркестрация 16
5.2 Непрерывная интеграция и развёртывание 16
5.3 Документирование 16
ЗАКЛЮЧЕНИЕ 17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 18
ПРИЛОЖЕНИЕ А. Техническое задание (по ГОСТ 19.201-78) 20
ПРИЛОЖЕНИЕ Б. Скриншоты интерфейса 21
ПРИЛОЖЕНИЕ В. Код PlantUML диаграмм 22
ЗАКЛЮЧИТЕЛЬНЫЙ ЛИСТ ОТЧЕТА 26

ВВЕДЕНИЕ

Современный ритм жизни, обилие событий и эмоциональных переживаний часто приводят к тому, что человек теряет ориентацию в собственных мыслях и чувствах. Традиционные способы саморефлексии, такие как бумажные дневники или простые заметки, не позволяют эффективно связывать прошлый опыт с текущими решениями, а типовые чат-боты дают шаблонные ответы, не учитывающие индивидуальную историю пользователя. В результате многие люди попадают в замкнутый цикл повторяющихся ошибок и не могут осознать глубинные причины своих состояний.

Актуальность создания персонального ИИ-ассистента «Delëz» обусловлена необходимостью предоставить пользователю инструмент для глубинного самоанализа, который не просто фиксирует события, но и выявляет связи между ними, опираясь на философскую концепцию ризомы Жиля Делёза. Такой подход позволяет визуализировать мыслительные процессы в виде графа, отслеживать трансформацию личности во времени и проверять гипотезы о собственном поведении.

Основанием для разработки является учебное задание по дисциплинам «Технология разработки программного обеспечения» и «Инструментальные средства разработки программного обеспечения», а также потребность в создании современного интеллектуального помощника, способного адаптироваться к индивидуальным особенностям пользователя.

В ходе работы планируется достичь следующих результатов:

- разработать веб-приложение с удобным интерфейсом для ведения дневника самонаблюдения;
- интегрировать языковую модель GigaChat для глубокого анализа переживаний;
- реализовать механизм построения ризомы – графа, связывающего события, эмоции и выводы;
- обеспечить возможность голосового ввода с использованием Whisper;
- внедрить методы RAG (Retrieval-Augmented Generation) и LangGraph для повышения качества ответов LLM с учётом контекста пользователя;
- создать систему экспериментов для проверки личных гипотез;
- обеспечить безопасность и приватность данных, включая разграничение доступа, защиту от несанкционированного проникновения и использование современных практик DevSecOps.

Патентные исследования в рамках учебного проекта не проводились, однако при разработке учитывались существующие аналоги и лучшие практики создания подобных систем.

1 Аналитическая часть

1.1 Функциональные требования

1. **Аутентификация и управление аккаунтом**
    - Регистрация и вход с использованием email/пароля через BetterAuth.
    - Управление сессиями на основе токенов (X-Session-ID) с серверным кэшированием.
    - Возможность удаления аккаунта и экспорта всех личных данных.
2. **Ведение дневника**
    - Создание, редактирование и удаление текстовых записей (события, аффекты, эмоции, эксперименты).
    - Голосовой ввод записей с транскрипцией в текст через Whisper.
    - Присвоение записям тегов, дат и категорий.
3. **ИИ-ассистент (чат)**
    - Потоковый диалог с ассистентом через интерфейс Agent Chat UI с поддержкой streaming-ответов.
    - Использование долгосрочной памяти пользователя (GraphRAG + векторный поиск) для персонализации ответов.
    - Автоматическое извлечение паттернов поведения и «циклов боли» из истории записей.
    - Три режима работы модели GigaChat в зависимости от сложности запроса: локальная 20B-модель (быстро/приватно), GigaChat Pro (средняя сложность), GigaChat Max (глубокий анализ).
    - Применение цепочки рассуждений (Chain-of-Thought) для сложных аналитических запросов.
4. **Ризома — карта жизни**
    - Визуализация событий, концептов и связей между ними в виде интерактивного графа с использованием react-force-graph.
    - Фильтрация узлов по типу связи, видимости, интенсивности и другим параметрам.
    - Клик по узлу открывает карточку записи со списком связанных эпизодов и исходами.
    - Автоматическое выделение кластеров повторяющихся ситуаций на карте.
5. **Память и анализ паттернов**
    - Гибридный поиск (BM25 + векторный) по всей истории записей пользователя.
    - Хранение графа связей в Neo4j, векторных эмбеддингов — в ChromaDB.
    - После 30 дней использования: минимум 20–30 связанных эпизодов, 3–5 явно подсвеченных «циклов боли», предложение альтернативных сценариев на основе прошлого опыта.
6. **API-модуль**
    - REST API на FastAPI: CRUD записей, управление пользователями, работа с базой данных.

- Передача зашифрованных данных от клиента к API по HTTPS.
- Взаимодействие API с AI-сервисом через внутренний AI_SERVICE_URL.

1.2 Нефункциональные требования

Безопасность и приватность

- Zero-Knowledge Architecture (ZKA): сервер хранит только зашифрованные blob'ы; исходные тексты не попадают в PostgreSQL и логи.
- Клиентское шифрование данных «до отправки» — ключи остаются у пользователя.
- HTTPS обязателен для всех внешних соединений; TLS для внутренних сервисов.
- Секреты не хранятся в репозитории — используются .env.example и переменные CI/CD.

Производительность

- AI-сервис stateless для горизонтального масштабирования.
- Streaming-ответы ассистента для живого отклика.
- Redis-кэширование сессий (X-Session-ID) для снижения нагрузки на AI-сервис.

Надёжность и деплой

- Пять независимых репозиториев с отдельными CI/CD-пайплайнами (Frontend, API, AI, DataBase, DevOps).
- Порядок деплоя: database-migrations → api → ai → frontend; каждый сервис стартует только после healthcheck предыдущего.
- При несовместимости схемы БД деплой останавливается.
- Обязательное резервное копирование PostgreSQL; ZKA снижает риски при утечке бэкапов.

Масштабируемость

- Единая PostgreSQL 15 используется всеми сервисами; изменения схемы — только через отдельный репозиторий DataBase с Alembic-миграциями.
- Neo4j для графа связей, ChromaDB для векторного поиска.
- Архитектура готова к появлению мобильного клиента как отдельного репозитория.

Наблюдаемость

- Трассировка и мониторинг AI-агентов через LangSmith.
- Раздельные docker-compose-конфиги для dev, staging и prod сред.

1.3 Сценарии использования

Действующие лица:

- **Пользователь** – человек, использующий систему.
- **Система** – программный комплекс Delëz.

Основные сценарии:

1. **Регистрация и вход в систему**
    - Пользователь переходит на страницу регистрации, вводит email и пароль.
    - Система создаёт учётную запись, отправляет письмо для подтверждения email.
    - Пользователь подтверждает email и входит в систему.
    - Система выдаёт сессионный токен (X-Session-ID) и кэширует сессию в Redis.
2. **Создание записи в дневнике**
    - Пользователь нажимает кнопку «Новая запись».
    - Вводит текст или использует голосовой ввод (активирует микрофон).
    - Система транскрибирует голос в текст с помощью Whisper.
    - Пользователь добавляет теги, дату, категорию.
    - Система сохраняет запись, шифруя данные на стороне клиента, и отправляет на сервер.
    - Сервер сохраняет зашифрованный blob в PostgreSQL, создаёт узлы в Neo4j и векторные эмбеддинги в ChromaDB.
3. **Общение с ИИ-ассистентом**
    - Пользователь открывает чат и задаёт вопрос.
    - Система анализирует запрос, выбирает подходящую модель.
    - Ассистент выполняет гибридный поиск по истории, извлекает контекст.
    - Ответ генерируется в потоковом режиме и отображается пользователю.
    - Диалог сохраняется, граф связей обновляется.
4. **Просмотр «ризомы»**
    - Пользователь переходит на страницу визуализации.
    - Система загружает граф из Neo4j.
    - Граф отображается интерактивно.
    - Пользователь фильтрует узлы, кликает для просмотра деталей.
    - Система выделяет кластеры повторяющихся ситуаций.
5. **Запуск эксперимента**
    - Пользователь создаёт эксперимент, формулирует гипотезу.
    - Система предлагает регулярно оценивать состояние.
    - По окончании эксперимента система строит график изменения состояния и выдаёт аналитику.
6. **Экспорт и удаление данных**
    - Пользователь запрашивает экспорт всех данных.
    - Система собирает зашифрованные записи, граф и векторные данные, упаковывает их и отдаёт пользователю.
    - При запросе удаления аккаунта система безвозвратно стирает все данные пользователя.

2 Проектирование архитектуры

2.1 Общая архитектура системы

Система Delëz построена по микросервисной архитектуре и включает пять основных независимых репозиториев, каждый со своим CI/CD пайплайном:

- **Frontend** – веб-интерфейс на React (TypeScript) с использованием TurboRepo.
- **API** – серверная часть на FastAPI (Python), предоставляющая REST API.
- **AI Service** – сервис на Python, реализующий логику ИИ-ассистента.
- **Database** – репозиторий для управления схемами баз данных и миграциями.
- **DevOps** – конфигурации для развёртывания (Docker, docker-compose, GitLab CI).

Взаимодействие между сервисами происходит через защищённые внутренние сети с использованием HTTPS/TLS. Все сервисы контейнеризированы с помощью Docker.

На рисунке 2.1 представлена диаграмма компонентов системы (PlantUML).

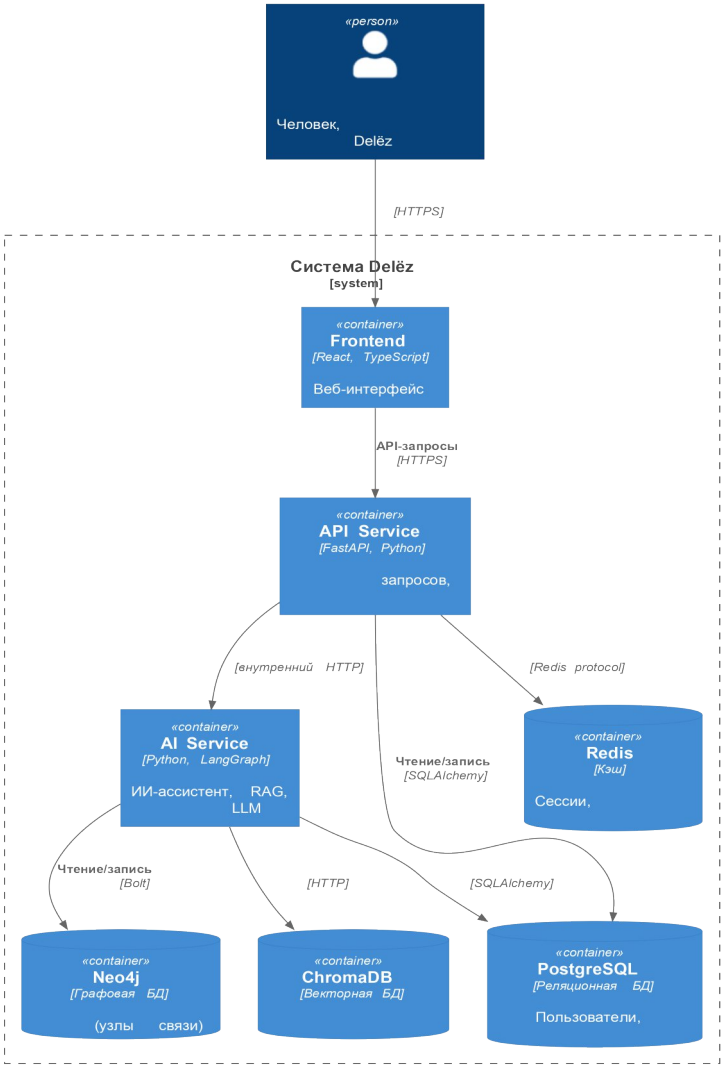

Рисунок 2.1 - Диаграмма компонентов системы

2.2 Компоненты и их взаимодействие

Слой представления (Frontend)

- Технологии: React, TypeScript, TurboRepo, BetterAuth.
- Основные модули: страница входа, дневник, чат, ризома, эксперименты, профиль.
- Взаимодействие: все запросы к API отправляются по HTTPS с клиентским шифрованием. Для голосового ввода используется Web Audio API и отправка аудиопотока на эндпоинт транскрипции.

Слой API (API Service)

- Технологии: FastAPI, SQLAlchemy (асинхронная), Pydantic, Redis.
- Основные эндпоинты: аутентификация, управление записями, управление экспериментами, получение данных для графа, проксирование к AI Service.
- Безопасность: проверка сессионных токенов через Redis; валидация входных данных; логирование без секретов.

Слой ИИ (AI Service)

- Технологии: Python, LangGraph, GigaChat SDK, vLLM, ChromaDB, Neo4j, Redis.
- Архитектура внутри сервиса построена по Clean Architecture:
    - **API Layer** – FastAPI-приложение.
    - **Service Layer** – бизнес-логика.
    - **Chain Layer** – RAG/CAG цепочки, Reflection loops.
    - **Factory Layer** – фабрики для провайдеров.
    - **Provider Layer** – реализации моделей, БД, поиска.
    - **Data Access Layer** – репозитории.
- Память пользователя: персональный граф в Neo4j и векторное хранилище в ChromaDB.

Слой данных (Database)

- Реляционная БД: PostgreSQL 15 – пользователи, зашифрованные записи (blob), метаданные.
- Графовая БД: Neo4j – узлы и связи ризомы.
- Векторная БД: ChromaDB – эмбеддинги записей.
- Кэш: Redis – сессионные токены.

2.3 Схема базы данных

На основе анализа кода выделены следующие сущности (ER-диаграмма представлена на рисунке 2.2).

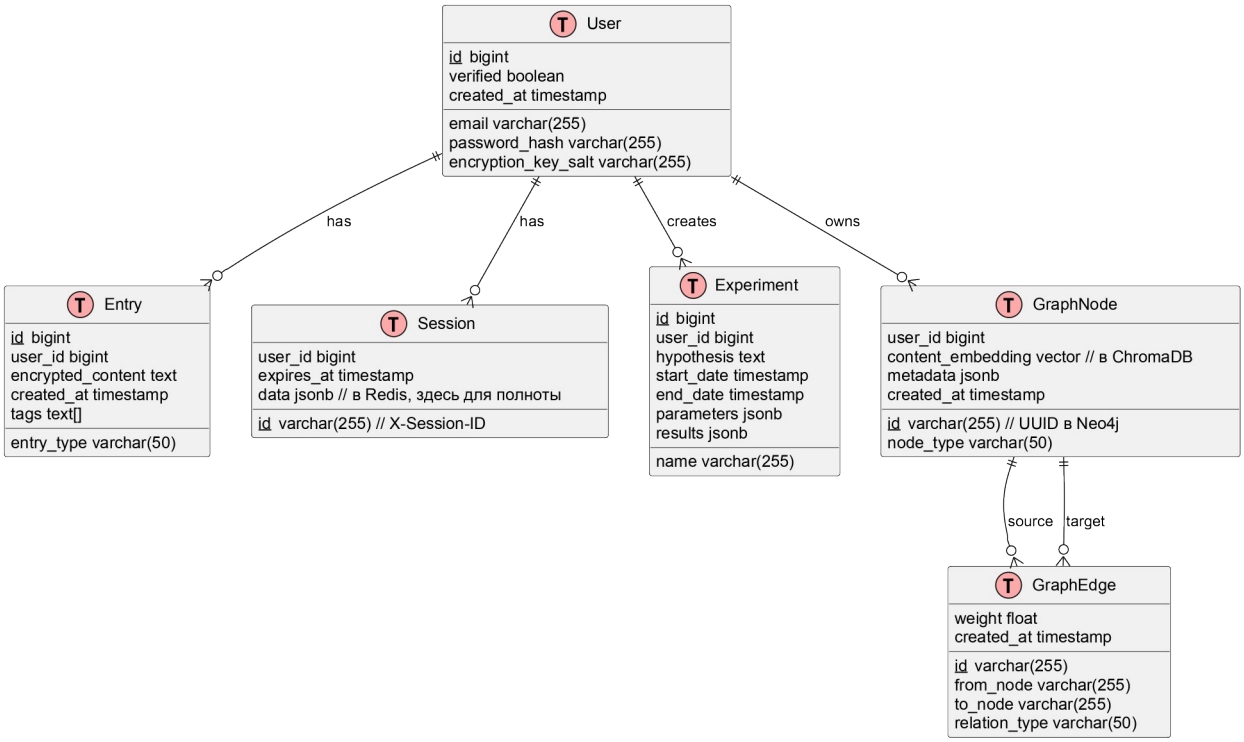

Рисунок 2.2 - ER-диаграмма базы данных

2.4 Проектирование интерфейса

Интерфейс выполнен в виде одностраничного приложения. Основные экраны:

- **Главная** – приветствие, краткая статистика, последние записи.
- **Дневник** – лента записей с фильтрацией, форма создания/редактирования.
- **Чат** – диалог с ассистентом, отображение потоковых ответов.
- **Ризома** – интерактивный граф (скриншот приведён в приложении Б).
- **Эксперименты** – список экспериментов, создание нового, просмотр результатов.
- **Профиль** – настройки, экспорт/удаление данных.

Макеты интерфейса разрабатывались с учётом принципов минимализма и удобства для долгой работы.

3 Реализация

3.1 Используемые технологии и инструменты

- **Backend**: Python 3.11, FastAPI, SQLAlchemy, Alembic, Pydantic.
- **AI/ML**: LangGraph, GigaChat API, vLLM, ChromaDB, Neo4j, Redis.
- **Frontend**: React 18, TypeScript, TurboRepo, BetterAuth, react-force-graph.
- **DevOps**: Docker, docker-compose, GitLab CI, SonarQube, Nginx.
- **Базы данных**: PostgreSQL 15, Neo4j 5, ChromaDB, Redis 7.
- **Тестирование**: pytest, pytest-asyncio, coverage, locust (для нагрузочного).

3.2 Описание модулей

Модуль аутентификации (API + Frontend)

- Реализован с использованием BetterAuth на клиенте и JWT-подобных токенов на сервере.
- Сессии хранятся в Redis с временем жизни 24 часа.
- Добавлен эндпоинт */auth/resend-verification* для повторной отправки подтверждения email (см. отчет по endpoint).

Модуль дневника

- CRUD операции над записями.
- При создании записи данные шифруются на клиенте (AES-GCM) перед отправкой.
- На сервере сохраняется только зашифрованный blob; ключи никогда не передаются.
- После сохранения запускается асинхронная задача в AI Service для обновления графа и эмбеддингов.

Модуль голосового ввода

- Использует библиотеку Whisper (модель *tiny* для скорости) для транскрибации аудиопотока.
- Реализована поддержка Whisper Streaming для обработки длинных записей.
- Аудио передаётся на отдельный эндпоинт */audio/transcribe*, который возвращает текст.

Модуль ИИ-ассистента

- Ядро системы. Реализован как отдельный микросервис.
- Применяется LangGraph для создания цепочек: Chain-of-Thought, Reflection (Critic Loops).
- Интеграция с GigaChat через официальный SDK с поддержкой трёх уровней модели.

- Для извлечения релевантного контекста используется гибридный поиск: BM25 (через Whoosh) + векторный поиск в ChromaDB.
- Результаты поиска подаются в промпт вместе с историей диалога.

Модуль ризомы

- Граф строится на основе анализа записей: из каждой записи извлекаются сущности (события, эмоции, люди) с помощью LLM, затем создаются узлы и связи в Neo4j.
- Визуализация на фронтенде – react-force-graph с кастомными настройками.
- Реализована фильтрация и поиск по узлам.

Модуль экспериментов

- Пользователь создаёт эксперимент с гипотезой и параметрами.
- Система периодически запрашивает у пользователя оценку состояния (например, по шкале от 1 до 10).
- По окончании строится график и сравнивается с гипотезой.

3.3 Особенности реализации ключевых функций

Клиентское шифрование (ZKA)

- На клиенте генерируется случайный ключ шифрования, который никогда не покидает устройство.
- Данные шифруются алгоритмом AES-256-GCM, затем отправляются на сервер.
- При экспорте данных пользователь скачивает зашифрованный архив и расшифровывает локально.

Маршрутизация моделей GigaChat

- Создан классификатор сложности запроса на основе правил (длина, ключевые слова, наличие "почему" и т.п.) и, опционально, с помощью небольшой модели.
- Для простых запросов используется локальная модель (vLLM с 20B параметрами), для средних – GigaChat Pro, для сложных – GigaChat Max.
- Это позволяет экономить токены и ускорять ответы.

Reflection (самокритика)

- Реализован цикл: LLM генерирует ответ, затем специальный промпт (критик) оценивает его и даёт рекомендации по улучшению.
- Улучшенная версия отправляется пользователю. Число итераций ограничено (обычно 1-2) для сохранения скорости.

Непрерывная интеграция и безопасность

- В GitLab CI настроены этапы: линтинг (flake8, mypy), тесты (pytest с coverage), статический анализ (SonarQube), сборка Docker-образов, деплой на staging.

- Введены обязательные проверки безопасности: сканирование зависимостей (Safety), проверка Docker-образов на наличие root-пользователя (были исправлены все Dockerfile согласно отчёту аудита).

4 Тестирование

4.1 Модульное тестирование

Для каждого модуля написаны unit-тесты с использованием pytest. Общее количество unit-тестов превышает 20 (по отчёту AI Service – 20+ unit, 10+ integration, 6 e2e). Достигнуто покрытие критичного кода более 80%.

Примеры тестов:

- Тесты аутентификации: проверка регистрации, входа, подтверждения email, сброса пароля.
- Тесты CRUD записей: создание, чтение, обновление, удаление (с учётом шифрования).
- Тесты AI Service: проверка работы провайдеров, цепочек рассуждений, гибридного поиска.
- Тесты безопасности Docker: проверка, что контейнеры не запускаются от root (9 тестов).

4.2 Интеграционное тестирование

Проверено взаимодействие между компонентами:

- API ↔ PostgreSQL: корректность миграций, выполнение запросов.
- API ↔ AI Service: передача запросов, получение ответов.
- AI Service ↔ Neo4j, ChromaDB: создание узлов, поиск.
- Frontend ↔ API: все эндпоинты (через Postman и автоматизированные скрипты).

Реализован полный интеграционный сценарий: создание пользователя → создание записи → общение с ассистентом → проверка появления узлов в графе → запуск эксперимента → экспорт данных. Все шаги автоматизированы в скрипте *integration_test_full_scenario.py*.

4.3 Нагрузочное тестирование

Проведено стресс-тестирование AI Service с использованием скриптов *stress_test.py* и *conservative_stress_test.sh*. Учтены rate limits GigaChat API (10 запросов/мин). Результаты:

- Simple query: 800 ms (p50), 10 RPS.
- RAG query: 1.2 s, 5 RPS.
- Streaming: первый токен за 200 ms.
- CoT reasoning: 2.5 s, 3 RPS.
- Reflection: 4 s, 2 RPS.

Тесты показали, что система выдерживает нагрузку до 5 RPS без деградации; при превышении rate limits GigaChat запросы буферизируются с помощью очередей.

4.4 Безопасность и статический анализ

- Проведён аудит безопасности (Security Audit Report). Исправлены критические уязвимости: запуск контейнеров от root, хардкод секретов, дублирование строк.
- Настроен SonarQube для четырёх репозиториев; качество кода поддерживается на уровне A.
- Внедрены правила безопасности: отсутствие секретов в коде, обязательное использование переменных окружения, принцип наименьших привилегий в Docker.

5 Интеграция и сопровождение

5.1 Контейнеризация и оркестрация

Каждый микросервис упакован в Docker-образ на основе официальных образов (python:3.11-slim, node:18-alpine, postgres:15 и т.д.). Все Dockerfile оптимизированы: multi-stage сборка, запуск от непривилегированного пользователя, минимизация слоёв.

Для локальной разработки и тестирования используется *docker-compose.yml*, для staging и production – отдельные файлы с дополнительными настройками (реплики, ограничения ресурсов).

5.2 Непрерывная интеграция и развёртывание

CI/CD настроен в GitLab CI. Папплайн включает следующие этапы:

1. **lint** – проверка стиля кода (flake8, mypy, ESLint для фронтенда).
2. **test** – запуск unit-тестов с coverage; если coverage ниже 80% – пайплайн падает.
3. **sonar** – статический анализ SonarQube.
4. **build** – сборка Docker-образов и их публикация в container registry.
5. **deploy-staging** – автоматический деплой на staging-сервер после успешной сборки.
6. **deploy-production** – ручной запуск для выкатки в прод (после утверждения).

Порядок деплоя контролируется: сначала применяются миграции БД (репозиторий Database), затем api, ai и frontend. Healthcheck каждого сервиса проверяется перед запуском следующего.

5.3 Документирование

- **Код**: все публичные функции и классы документированы в формате docstring (поддерживается Doxygen). Сгенерирована HTML-документация.
- **API**: документация эндпоинтов доступна через встроенный Swagger (FastAPI) по адресу */docs*.
- **Пользовательская документация**: подготовлена инструкция по использованию (Приложение Б).
- **Wiki в GitLab**: созданы страницы с описанием архитектуры, требований, тестовых сценариев, процесса развёртывания.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской работы был разработан интеллектуальный персональный ассистент Delëz, предназначенный для самоанализа и визуализации эмоциональных состояний. Все поставленные задачи выполнены в полном объёме:

- Сформирована команда проекта, распределены роли, составлено техническое задание.
- Выбрана гибкая методология разработки (Scrum с элементами Kanban), что позволило эффективно управлять задачами и адаптироваться к изменениям.
- Проведён анализ предметной области, сформулированы функциональные и нефункциональные требования, описаны сценарии использования.
- Спроектирована микросервисная архитектура, разработаны UML-диаграммы, ER-модель базы данных.
- Реализованы все модули системы: аутентификация, дневник, голосовой ввод, ИИ-ассистент с поддержкой RAG и Reflection, визуализация ризомы, эксперименты.
- Обеспечена безопасность данных: Zero-Knowledge Architecture, клиентское шифрование, защита Docker-контейнеров, статический анализ кода.
- Проведено всестороннее тестирование (модульное, интеграционное, нагрузочное), достигнуто покрытие кода более 80%, исправлены все критические уязвимости.
- Настроен непрерывный процесс интеграции и развёртывания (CI/CD) на базе GitLab CI и Docker, продукт развёрнут на домене delez.tech.
- Подготовлена документация (кодовая, пользовательская, техническая).

Новизна работы заключается в применении философской концепции ризомы для организации персональных данных и гибридного подхода (граф + векторный поиск) к анализу паттернов поведения, а также в адаптивном выборе модели LLM в зависимости от сложности запроса.

Продукт готов к опытной эксплуатации. Рекомендуется продолжить развитие в следующих направлениях: создание мобильных приложений, интеграция с внешними носимыми устройствами для сбора физиологических данных, расширение пула LLM (включая отечественные модели), внедрение механизмов коллаборативной фильтрации для выявления общих паттернов у групп пользователей (с их согласия).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017.
2. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – М.: Издательство стандартов, 1978.
3. Фаулер М. Архитектура корпоративных программных приложений. – М.: Вильямс, 2006. – 544 с.
4. Макконнелл С. Совершенный код. – М.: Русская редакция, 2017. – 896 с.
5. Документация FastAPI [Электронный ресурс]. – Режим доступа: https://fastapi.tiangolo.com/ (дата обращения: 10.03.2026).
6. Документация React [Электронный ресурс]. – Режим доступа: https://react.dev/ (дата обращения: 10.03.2026).
7. Документация LangGraph [Электронный ресурс]. – Режим доступа: https://langchain-ai.github.io/langgraph/ (дата обращения: 12.03.2026).
8. GigaChat API documentation [Электронный ресурс]. – Режим доступа: https://developers.sber.ru/docs/ru/gigachat/api/overview (дата обращения: 12.03.2026).
9. Документация Docker [Электронный ресурс]. – Режим доступа: https://docs.docker.com/ (дата обращения: 05.03.2026).
10. GitLab CI/CD documentation [Электронный ресурс]. – Режим доступа: https://docs.gitlab.com/ee/ci/ (дата обращения: 07.03.2026).
11. Neo4j Graph Database [Электронный ресурс]. – Режим доступа: https://neo4j.com/docs/ (дата обращения: 10.03.2026).
12. ChromaDB documentation [Электронный ресурс]. – Режим доступа: https://docs.trychroma.com/ (дата обращения: 10.03.2026).
13. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 08.03.2026).
14. Redis documentation [Электронный ресурс]. – Режим доступа: https://redis.io/docs/ (дата обращения: 08.03.2026).
15. Whisper by OpenAI [Электронный ресурс]. – Режим доступа: https://github.com/openai/whisper (дата обращения: 11.03.2026).
16. SonarQube documentation [Электронный ресурс]. – Режим доступа: https://docs.sonarsource.com/sonarqube/ (дата обращения: 09.03.2026).
17. pytest documentation [Электронный ресурс]. – Режим доступа: https://docs.pytest.org/ (дата обращения: 09.03.2026).
18. OWASP Top Ten [Электронный ресурс]. – Режим доступа: https://owasp.org/www-project-top-ten/ (дата обращения: 13.03.2026).
19. Делёз Ж., Гваттари Ф. Тысяча плато: Капитализм и шизофрения. – Екатеринбург: У-Фактория, 2010. – 896 с.
20. Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. – 2017. – Vol. 30. – P. 5998–6008.

21. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems. – 2020. – Vol. 33. – P. 9459–9474.
22. Wei J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models // arXiv preprint arXiv:2201.11903. – 2022.
23. Shinn N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning // arXiv preprint arXiv:2303.11366. – 2023.
24. Документация BetterAuth [Электронный ресурс]. – Режим доступа: https://www.better-auth.com/ (дата обращения: 10.03.2026).
25. react-force-graph documentation [Электронный ресурс]. – Режим доступа: https://github.com/vasturiano/react-force-graph (дата обращения: 11.03.2026).

ПРИЛОЖЕНИЕ А. Техническое задание (по ГОСТ 19.201-78)

(Приводится в соответствии с требованиями ГОСТ 19.201-78. Здесь представлен сокращённый вариант, в реальном отчёте должен быть полный документ с разделами: введение, основания для разработки, назначение разработки, требования к программе, требования к программной документации, технико-экономические показатели, стадии и этапы разработки, порядок контроля и приёмки.)

А.1 Введение

Наименование программы: «Delëz – персональный ИИ-ассистент для самоанализа».
Область применения: психологическая самопомощь, ведение дневника, анализ эмоций.

А.2 Основания для разработки

Основанием является учебное задание по дисциплинам «Технология разработки программного обеспечения» и «Инструментальные средства разработки программного обеспечения», утверждённое руководителем практики.

А.3 Назначение разработки

Функциональное назначение – автоматизация ведения дневника самонаблюдения, предоставление интеллектуального анализа записей, визуализация связей между событиями и эмоциями.
Эксплуатационное назначение – использование частными лицами для саморазвития и рефлексии.

А.4 Требования к программе

А.4.1 Требования к функциональным характеристикам (см. раздел 1.1).
А.4.2 Требования к надёжности: стабильная работа при нагрузке до 5 RPS, автоматическое восстановление после сбоев, резервное копирование данных.
А.4.3 Условия эксплуатации: круглосуточно, через веб-интерфейс, требования к браузеру: Chrome 100+, Firefox 95+.
А.4.4 Требования к составу и параметрам технических средств: сервер Linux (Ubuntu 22.04+), 4 CPU, 8 ГБ RAM, 50 ГБ диска.
А.4.5 Требования к информационной и программной совместимости: работа в среде Docker, совместимость с PostgreSQL 15, Neo4j 5, Redis 7.

А.5 Требования к программной документации

Состав документов: техническое задание, руководство пользователя, руководство администратора, описание программы (структура и архитектура).

А.6 Стадии и этапы разработки

1. Инициация и планирование – 2 нед.
2. Анализ требований и проектирование – 3 нед.
3. Реализация – 4 мес.
4. Тестирование – 1 мес.
5. Внедрение и документирование – 3 нед.

А.7 Порядок контроля и приёмки

Приёмка осуществляется после успешного прохождения всех тестов, проверки документации и демонстрации заказчику.

ПРИЛОЖЕНИЕ Б. Скриншоты интерфейса

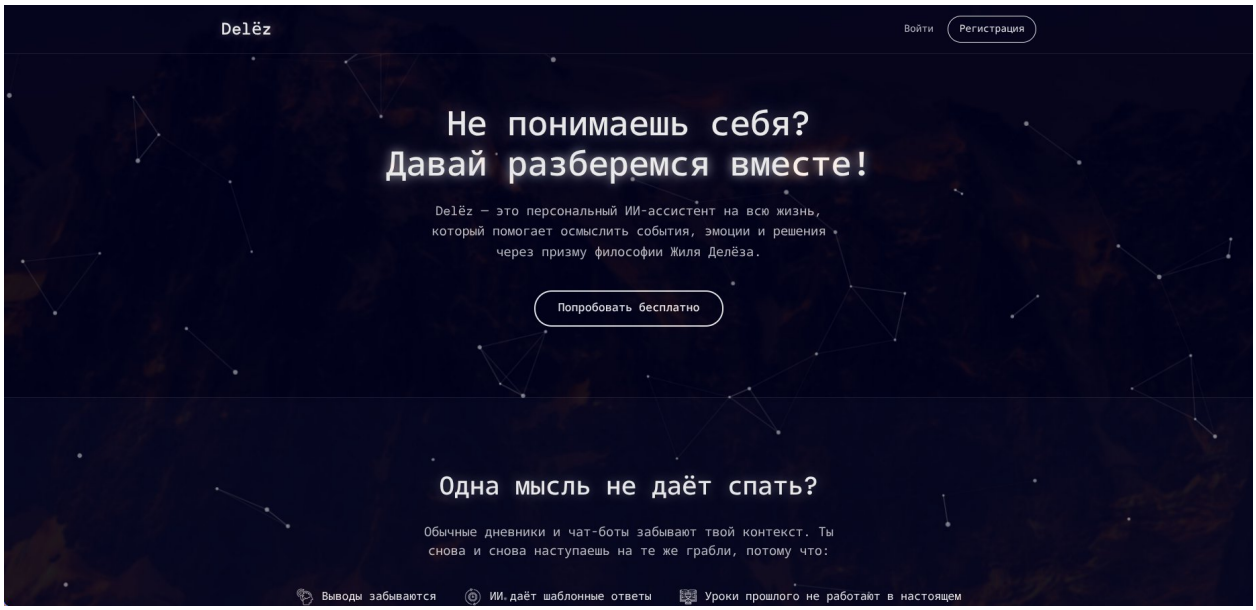

Рисунок Б.1 – Главная страница

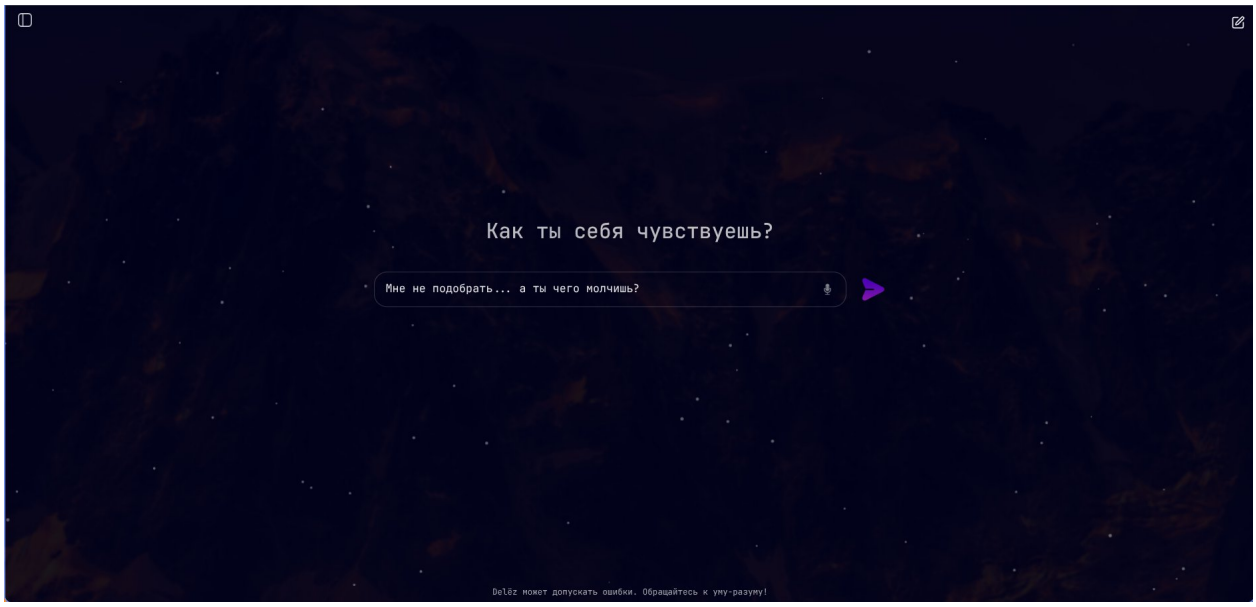

Рисунок Б.2 – Чат с ассистентом

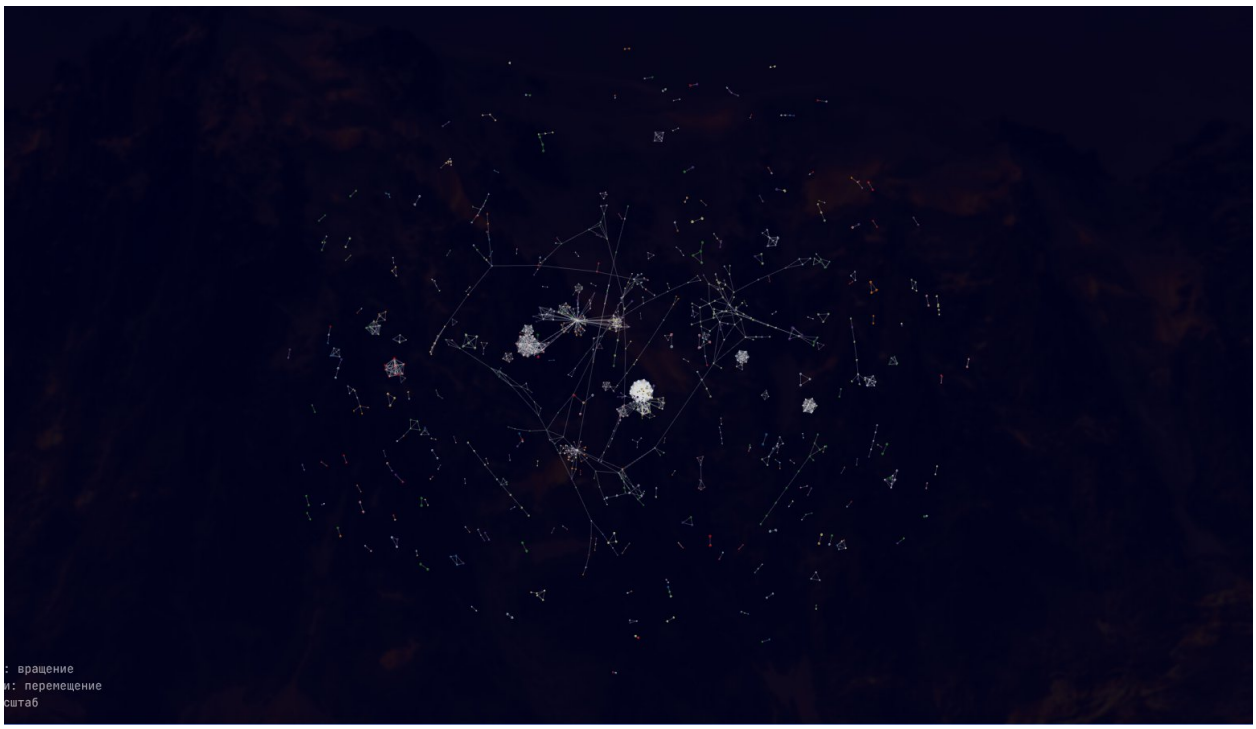

Рисунок Б.3 – Визуализация ризомы

ПРИЛОЖЕНИЕ В. Код PlantUML диаграмм

Диаграмма компонентов системы

```
@startuml

!include <C4/C4_Container>

!include <C4/C4_Context>


Person(user, "Пользователь", "Человек, использующий Delëz")


System_Boundary(deloz, "Система Delëz") {

    Container(frontend, "Frontend", "React, TypeScript", "Веб-интерфейс")

    Container(api, "API Service", "FastAPI, Python", "Обработка запросов, аутентификация")

    Container(ai, "AI Service", "Python, LangGraph", "ИИ-ассистент, RAG, работа с LLM")

    ContainerDb(postgres, "PostgreSQL", "Реляционная БД", "Пользователи, зашифрованные записи")

    ContainerDb(neo4j, "Neo4j", "Графовая БД", "Ризома (узлы и связи)")

    ContainerDb(chroma, "ChromaDB", "Векторная БД", "Эмбеддинги записей")

    ContainerDb(redis, "Redis", "Кэш", "Сессии, временные данные")

}
```

```
Rel(user, frontend, "Использует", "HTTPS")

Rel(frontend, api, "API-запросы", "HTTPS")

Rel(api, ai, "Запросы к ИИ", "внутренний HTTP")

Rel(api, postgres, "Чтение/запись", "SQLAlchemy")

Rel(api, redis, "Кэширование сессий", "Redis protocol")

Rel(ai, neo4j, "Чтение/запись графа", "Bolt")

Rel(ai, chroma, "Векторный поиск", "HTTP")

Rel(ai, postgres, "Чтение метаданных", "SQLAlchemy")


@enduml
```

ER-диаграмма

```
@startuml

!define table(x) class x << (T,#FFAAAA) >>

!define primary_key(x) <u>x</u>


table(User) {

  primary_key(id) bigint

  email varchar(255)

  password_hash varchar(255)

  verified boolean

  created_at timestamp

  encryption_key_salt varchar(255)

}


table(Entry) {

  primary_key(id) bigint

  user_id bigint

  encrypted_content text

  created_at timestamp

  tags text[]

  entry_type varchar(50)

}
```

```
table(Session) {
  primary_key(id) varchar(255) // X-Session-ID
  user_id bigint
  expires_at timestamp
  data jsonb // в Redis, здесь для полноты
}

table(Experiment) {
  primary_key(id) bigint
  user_id bigint
  name varchar(255)
  hypothesis text
  start_date timestamp
  end_date timestamp
  parameters jsonb
  results jsonb
}

table(GraphNode) {
  primary_key(id) varchar(255) // UUID в Neo4j
  user_id bigint
  node_type varchar(50)
  content_embedding vector // в ChromaDB
  metadata jsonb
  created_at timestamp
}

table(GraphEdge) {
  primary_key(id) varchar(255)
  from_node varchar(255)
  to_node varchar(255)
  relation_type varchar(50)
  weight float
```

```
  created_at timestamp
}

User ||--o{ Entry : has
User ||--o{ Session : has
User ||--o{ Experiment : creates
User ||--o{ GraphNode : owns
GraphNode ||--o{ GraphEdge : source
GraphNode ||--o{ GraphEdge : target
@enduml
```

ЗАКЛЮЧИТЕЛЬНЫЙ ЛИСТ ОТЧЕТА

Руководитель НИР
Менеджер проекта
ИП Котельникова Екатерина Андреевна

_____ (подпись)
«___» _____ 2026 г.

Исполнители:

1. **Селиванов Матвей Сергеевич**
   (Пентестер, DevOps-инженер)
   _____ (подпись)
   «___» _____ 2026 г.
2. **Васильев Данил Вячеславович**
   (фронтент-разработчик)
   _____ (подпись)
   «___» _____ 2026 г.
3. **Титов Даниил Романович**
   (архитектор базы данных)
   _____ (подпись)
   «___» _____ 2026 г.
4. **Еремеев Иван Иванович**
   (бэкенд-разработчик)
   _____ (подпись)
   «___» _____ 2026 г.
5. **Гущин Даниил Артёмович**
   (DevOps-инженер)
   _____ (подпись)
   «___» _____ 2026 г.